World-space Labels & Interaction System

Contents

- Files
- How It Fits Together
- Adding a Label to an Object
- Making Something Interactable
- Doors
- Gotchas

How the player sees and uses things in the world:

- **World labels:** floating names and interaction prompts that fade in as the player gets close.
- **Interaction:** how the interact key reaches the object the player is looking at.
- **Doors:** doors with keys, one-way doors and boss locks.

See also: Enemy Encounter & Spawning (boss doors), Shaders & VFX (dissolve for magic locks), Audio Manager.

Files

File	Job
`WorldLabel.cs`	Marks where a label goes and what kind it is.
`WorldLabelPool.cs`	Spawns, moves, turns and fades the label objects.
`WorldLabelOverlay.cs`	Makes the label prefab draw on top of walls.
`InteractionFocusController.cs`	Finds nearby labels each frame and sends the interact key to the aimed object.
`IInteractable.cs`, `InteractionPrompt.cs`, `InteractionContext.cs`	The interaction contract.
`Raycaster.cs`	Shared aim ray from the camera, used by interaction and the gun.
`Door.cs`, `DoorLock.cs`	Doors and their locks.

How It Fits Together

Labels and interaction are separate:

- **Labels** work by **proximity**: everything near the player gets a label.
- **Interaction** works by **aim**: only the object under the crosshair responds to the interact key.

The one link is that a label in **Interaction Prompt** mode asks its object what to say (*Open*, *Unlock*, *Locked*...), so the prompt always matches what interacting will do.

Adding a Label to an Object

1. Make sure the object's collider is on a layer in the controller's **Interactable Mask**.
2. Add a **child** object where the label should appear, and add **WorldLabel** to it.
3. Pick a **Mode**:
    - **Floating Label:** shows **Display Name** and turns to face the camera. For names and signs.
    - **Interaction Prompt:** shows the object's prompt, e.g. `[E] Open`. Lies flat against the object, facing the same way as the label's **parent**, and flips to face whichever side the player is on. The object needs an `IInteractable` (on the collider or a parent).

Labels fade from invisible at **Proximity Radius** to fully visible at **Label Full Opacity Distance**, and hide if something on the **Obstruction Mask** is in the way. All labels hide while the Grimoire is open.

Label prefab

The pool's **Label Prefab** needs a **World Space Canvas** with a **CanvasGroup** on the root, a **TMP_Text** in its children, and **WorldLabelOverlay** so it isn't hidden inside walls.

> **In code:** each frame the controller calls `labelPool.Show(id, position, rotation, text, alpha, mode)` for every visible label, then `labelPool.Flush()`, which fades out anything not shown this frame. Fading uses unscaled time so it still works while paused. `labelPool.SetVisible(false)` hides everything at once.

Making Something Interactable

Implement `IInteractable`:

```
// Summary: Example lever that toggles when interacted with.
public class Lever : MonoBehaviour, IInteractable
{
    private bool pulled;

    public void Interact(InteractionContext context)
    {
        pulled = !pulled;
    }

    public InteractionPrompt ResolvePrompt(InteractionContext context)
    {
        return new InteractionPrompt
        {
            label = pulled ? "Reset" : "Pull",
            actionName = "Collect"   // input action shown as the key
        };
    }
}
```

- `ResolvePrompt` runs **every frame** the label is visible, so keep it cheap and don't change anything in it.
- An empty `label` hides the prompt. An empty `actionName` shows the text without a key (used for *Locked*).
- `InteractionContext` holds the player's `Inventory`. Add fields to it (filled in `InteractionFocusController.Start`) if interactables need more information about the player.

The interact key (`Collect` action) fires **on release**, on the object the shared `Raycaster` is hitting, within **Interaction Range**.

> **In code:** the aim check uses `GetComponent<IInteractable>()` on the exact collider hit, not its parents. The label lookup is more forgiving and does check parents. Anything can read the aim ray through `Raycaster.Instance.Ray`, `.Hit` and `.HasHit`.

Doors

Basic door

Add **Door** to the door's pivot. Interacting toggles it between **Open** and **Closed**, rotating around Y by **Open Angle** / **Closed Angle**. When the player walks more than **Ajar Distance** away from an open door, it swings to **Ajar Angle** on its own (one-way doors close fully instead). The door's collider is off while it moves.

Locks

Add a **DoorLock** to each lock object (a padlock, a seal) and drag it into the door's **Door Locks** list.

DoorLock field	What it does
Required Item	The `CollectibleObject` key, checked against the player's inventory.
Consume Item On Unlock	Uses up the key.
Lock Visuals	Hidden once unlocked.
Unlock Dissolve Effect / **Destroy On Unlock**	For magic locks: dissolve, then destroy the lock object. Turn off **Destroy On Complete** on the DissolveEffect itself.
Locked Sound / Unlock Sound	Feedback.

Interacting with a locked door unlocks every lock the player has a key for. If any are still locked, it plays the *locked* sound. Once the last lock opens, the player interacts again to open the door.

The prompt shows **Unlock** if the player has at least one key, otherwise **Locked**.

One-way doors

Tick **Is One Way**. The accessible side is the door's forward side (a green gizmo arrow shows it), or the back with **Flip Accessible Side**. From the wrong side the door shows **Locked**.

Goal	Settings
Shortcut that becomes a normal door once opened	Is One Way + **Two Way Once Opened**
Free from one side, needs a key from the other. Normal door once unlocked.	Is One Way + **Accessible Side Bypasses Locks** + **Two Way When Unlocked** + a DoorLock
Only ever opens one way	Is One Way, everything else off

Boss locking

Boss spawners call `BossLock()` and `BossUnlock()` on their doors. While boss locked, a door closes and ignores all interaction.

> **In code:** `Open()`, `Close()`, `Ajar()` and `Toggle()` are public for scripted events and **ignore locks**, so check `DoorLock.IsLocked` first if a script should respect them. `DoorLock` also has `Unlock()` and `Lock()` to change it without a key.

Gotchas

- **Interaction Prompt labels must be on a child object**, since they face the same way as their parent.
- **The `IInteractable` must be on the collider the crosshair hits.**
- **No label showing?** Check the collider's layer is in the Interactable Mask, then check the Obstruction Mask.
- **Large trigger volumes can block the aim ray.** Keep room triggers off the Raycaster's layers.
- **Keys are matched by asset**, not by name.